American Sign Language Classification

A Computer Vision Approach for Real-Time ASL Recognition

Singparady Hok
Dept. of Information Technology
American University of Phnom Penh
Phnom Penh, Cambodia
2024568hok@aupp.edu.kh

Chipbong Te
Dept. of Information Technology
American University of Phnom Penh
Phnom Penh, Cambodia
2024572te@aupp.edu.kh

***Abstract*—This paper presents a real-time American Sign Language (ASL) recognition system designed to bridge the communication gap between signers and non-signers. By leveraging computer vision and deep learning, specifically Google MediaPipe for keypoint extraction and Spatio-Temporal Graph Convolutional Networks (ST-GCN) for gesture classification, the proposed system translates dynamic sign gestures into text. We utilized a composite dataset derived from Microsoft's ASL Citizen and MSASL to train and validate the model. Experimental results demonstrate that the system achieves an overall accuracy of approximately 85% on the combined test set, with notably higher performance on distinct static signs compared to complex dynamic gestures.**

***Keywords*—American Sign Language; Computer Vision; MediaPipe; ST-GCN; Deep Learning; Accessibility.**

I. INTRODUCTION

Those people who rely on American Sign Language often face communicative challenges when communicating with non-signers. It is easy to have miscommunications especially in learning institutions, hospitals, and even the workplace where some staff might not be well adept at the sign language. These barriers can be reduced through an automated American Sign Language recognition system that interprets signs and provides a real-time translation of them into an English text. Therefore, it promotes the accessibility of communicative means and makes the interaction between signers and non-signers more closer [1].

This issue is important as it helps to combine technology and accessibility. It shows how modern AI can be used to provide communication to the hearing impaired. Educationally, the project will allow using the concepts of pose estimation, video preprocessing, graph neural networks, and real-time inference [2]. In addition, it helps learners to understand how raw video data may be transformed into structured skeletal representations as graph images to be used in machine learning [3].

A. Objectives of the Work

- To apply computer vision and deep learning knowledge in a ASL recognition project.
- To collect or use video datasets and extract hand keypoints using MediaPipe.
- To train an ST-GCN model for ASL gesture classification based on skeleton data.
- To contribute to accessible technology that improves communication for deaf and hard-of-hearing users.

II. DATASETS

To ensure the robustness of the recognition system, we structured our experiments around two distinct dataset configurations.

A. Data Sources

- **Combined Dataset (ASL Citizen + MSASL):** This dataset aggregates samples from two major sources:
  - *ASL Citizen:* A large-scale crowdsourced dataset originally containing 2,731 classes [4].
  - *MSASL:* A real-world dataset scraped from web videos, originally containing 1,000 classes [6].

  We filtered both sources to focus on a common subset of classes to create a robust training set.
- **Google - Isolated Sign Language Recognition:** Sourced from the Kaggle competition (PopSign), this dataset serves as a separate benchmark. It originally contains 250 distinct sign classes [7]. We utilized a filtered subset of this dataset, selecting 20 classes (100 samples each) to evaluate model performance on difference dataset.

B. Dataset Splits

To effectively train and evaluate the model, we applied specific split ratios tailored to each dataset configuration:

- **Google - Isolated Sign Language Recognition:** For this benchmark dataset, we adhered to a standard split of **80% Training**, **10% Validation**, and **10% Testing**.
- **Combined Dataset (ASL Citizen + MSASL):** To ensure rigorous evaluation on our primary combined dataset, we utilized a split of **60% Training**, **20% Validation**, and **20% Testing**, allocating a larger portion to validation and testing to better assess generalization.

C. Preprocessing and Feature Extraction

The raw videos were recorded at 30 fps with varying lengths. We employed a three-step preprocessing pipeline using Google MediaPipe [3].

For the **Google - Isolated Sign Language Recognition** dataset, the data was originally provided as pre-extracted MediaPipe keypoints stored in .parquet files. To ensure data quality and consistency, we applied specific filtering criteria:

1. **Class Selection:** We filtered the dataset from its original 250 classes down to the 20 specific classes relevant to our study.
2. **Frame Filtering:** We selected samples with a duration between **20 and 60 frames**. This range was chosen to minimize noise and "polluted data" often found in very short or excessively long sequences.
3. **Target Frame:** All selected samples were processed to a uniform target length of **60 frames** for input into the model.

For the **Combined Dataset (ASL Citizen + MSASL)**, we implemented specific frame extraction strategies to optimize temporal resolution:

1. **Extraction:** We extracted the default 543 keypoints from each video sample. We tested two configurations: extracting a sequence of **60 frames** and a longer sequence of **128 frames**.
2. **Selection:** We filtered the keypoints down to **27 essential keypoints (11 pose + 8 right hand + 8 left hand)** representing the upper body and hands.
3. **Normalization & Padding:** The 27 keypoints were normalized relative to the midpoint between the shoulders (the center). To handle variable video lengths and missing detections, NaN values and frames shorter than the target length were zero-padded to ensure consistent input dimensions.

The final processed data is stored as '.npy' files for efficient training.

III. PROPOSED METHODS

A. Model Architecture

The core of our recognition system is a Spatio-Temporal Graph Convolutional Network (ST-GCN) implemented in TensorFlow/Keras [2]. The model processes skeletal data represented as a tensor of shape (N, C, T, V), where N is the batch size, C is the number of channels (coordinates), T is the number of frames (either 60 or 128 depending on the experiment), and $V = 27$ is the number of keypoints.

The architecture is built on a backbone of 10 ST-GCN blocks. Each block performs a spatial graph convolution followed by a temporal convolution (kernel size 9×1). The graph convolution uses an adjacency matrix A representing the skeletal connections (partitioned into centripetal, centrifugal, and self-links) to aggregate features from neighboring joints.

Table 1: ST-GCN Network Backbone Configuration

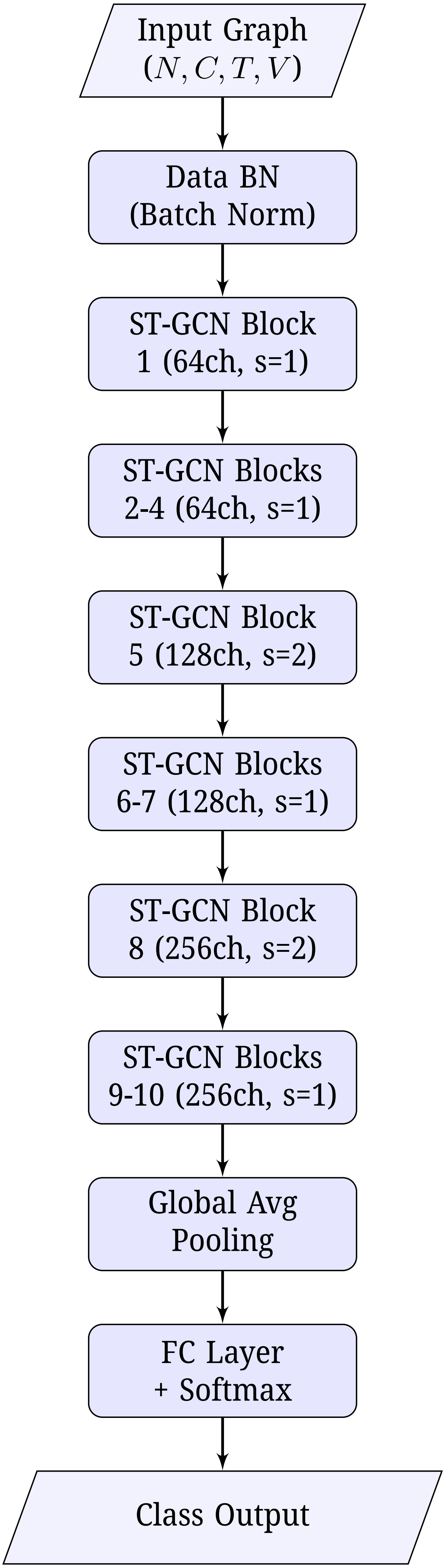
Block	In Ch.	Out Ch.	Stride	Notes
Input	C_{in}	64	1	Data Batch Norm
1	64	64	1	No Residual
2 – 4	64	64	1	Residual
5	64	128	2	Downsampling
6 – 7	128	128	1	Residual
8	128	256	2	Downsampling
9 – 10	256	256	1	Residual
Head	256	256	-	Global Avg Pool
Output	256	Classes	-	FC + Softmax

The network progressively increases the channel capacity from 64 to 256 while downsampling the temporal dimension in blocks 5 and 8. The final output is generated via global average pooling and a fully connected layer.

B. Training Configuration

The model was trained using the **Adam** optimizer with the following hyperparameters:

- **Learning Rate:** CosineDecay schedule, starting at 0.001 and decaying to a minimum of 0.00001.
- **Batch Size:** 32.
- **Epochs:** 100, with Early Stopping (patience: 40, min_delta: 0.001).
- **Loss Function:** Categorical cross-entropy with label smoothing (0.1).



C. Data Augmentation

To improve model robustness and invariance to viewing angles, we implemented a geometric augmentation pipeline applied on-the-fly during the training phase. We utilized two specific affine transformations on the spatial (x, y) coordinates of the skeleton joints:

- **Random Rotation:** We applied a 2D rotation around the origin with an angle θ sampled from a normal distribution $\mathcal{N}(0, 0.1)$, corresponding to approximately $\pm 5.7°$.
- **Random Shear:** We applied a linear shear transformation to simulate perspective distortion, where the shear factor is sampled from $\mathcal{N}(0, 0.1)$.

These transformations are chained using a Compose class and applied with a probability of 1.0 during training, while the validation and test sets remain unaugmented.

D. Design Rationale: Why ST-GCN?

We selected ST-GCN over traditional video-based CNNs (like I3D) for several reasons:

- **Suitability:** ST-GCN explicitly models the spatial graph structure of human joints and their temporal evolution, making it a natural fit for our keypoint-based data.
- **Data Efficiency:** CNNs like I3D require massive video datasets to learn appearance and motion features. ST-GCN operates on compact skeleton representations, making it far more efficient for our limited dataset (25 samples/class). Initial experiments with I3D yielded poor results due to overfitting [5].
- **Invariance:** ST-GCN focuses on motion and configuration, making it invariant to background, clothing, and lighting changes.
- **Interpretability:** The graph-based approach allows for easier debugging of joint importance compared to pixel-based methods.

IV. EXPERIMENTS AND RESULTS

A. Results

The results of our experiments are summarized in Table 2. We compared the performance of our ST-GCN model on two primary dataset configurations. First, we evaluated the model on the **Google - Isolated Sign Language Recognition** dataset (20 classes). Second, we evaluated the model on our **Combined Dataset (ASL Citizen + MSASL)**, focusing on the same subset of 20 classes. The combined dataset yielded an accuracy of 0.84 when using 128 frames. By further refining this combined dataset to use only 60 frames per video sample, we achieved our best performance with an accuracy of 0.85.

Table 2: Performance comparison between dataset configurations.

Dataset	Accuracy	Precision	Recall	F1-Score
Google	0.73	0.72	0.73	0.72
Combined (128)	0.84	0.86	0.84	0.84
Combined (60)	**0.85**	**0.86**	**0.85**	**0.85**

The confusion matrix (Figure 2) reveals a distinct pattern in performance. The model achieves high accuracy (approx. 86%) on classes 0-9. However, performance drops for classes 10-19, with significant confusion observed between Class 13, 14, and 17.

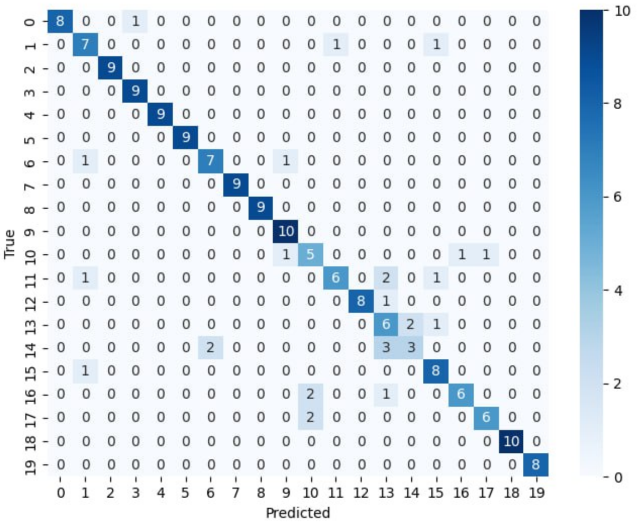


Figure 2: Confusion Matrix showing high accuracy for classes 0-9 and increased confusion in classes 10-19.

This discrepancy suggests that the model excels at the broad gestures found in the earlier classes but struggles with the fine-grained finger similarities present in the latter dynamic signs (e.g., Class 14).

V. CONCLUSION

This project successfully developed a real-time ASL recognition system. By combining MediaPipe with ST-GCN and refining the input data to a 60-frame window, we achieved a classification accuracy of 85% on our composite dataset. The system shows promise for accessible technology, though future work is needed to address the confusion between similar dynamic signs in the 10-19 class range.

VI. REFERENCES

[1] E. Vahdani and Y. Tian, “Deep Learning-based Isolated Sign Language Recognition: A Review,” *arXiv preprint arXiv:2203.15852*, 2022. https://doi.org/10.48550/arXiv.2203.15852

[2] S. Yan, Y. Xiong, and D. Lin, “Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018. https://doi.org/10.1609/aaai.v32i1.12328

[3] F. Zhang *et al.*, “MediaPipe Hands: On-device Real-time Hand Tracking,” *arXiv preprint arXiv:2006.10214*, 2020. https://arxiv.org/abs/2006.10214

[4] A. Desai *et al.*, “ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition,” *arXiv preprint arXiv:2304.05934*, 2024. https://arxiv.org/abs/2304.05934

[5] D. Li, C. Rodriguez, X. Yu, and H. Li, “Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1459–1469. https://doi.org/10.1109/WACV45572.2020.9093512

[6] H. R. V. Joze and O. Koller, “MS-ASL: A Large-Scale Data Set and Benchmark for Understanding

American Sign Language," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2019. https://doi.org/10.48550/arXiv.1812.01053

[7] T. Starner *et al.*, "PopSign ASL v1.0: An Isolated American Sign Language Dataset Collected via Smartphones," in *Proceedings of the NeurIPS Datasets and Benchmarks Track*, 2023.